



HASHDIT

HashDit

Smart Contract Security

ListaDao YieldAccount Audit

Security Audit Report

ListaDao

08 Sep 2026 - 11 Sep 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	08 Sep 2026 – 11 Sep 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/242
Commit	feat/yield-account

Smart Contract List

No.	Contract Name	Verdict	Details
1	PositionMigrator.sol	Medium	See Findings
2	YieldAccount.sol	Medium	See Findings

Findings

[M01] YIELD_ACCOUNT_OWNER can block forceMigrate after the migration deadline	
Contract	PositionMigrator.sol & YieldAccount.sol
Severity	Medium
Description	Description The forceMigrate() function in PositionMigrator is callable by the BOT role once migrationDeadline has passed, and its purpose is to move the hardcoded YIELD_ACCOUNT_OWNER's CDP position into the routed YieldAccount even when that owner does not cooperate.
	<pre>JavaScript function setMigratorAuthorization(address _migrator, bool enabled) external { require(msg.sender == OWNER hasRole(MANAGER, msg.sender), NotAuthorized()); require(_migrator != address(0), ZeroAddress()); require(_migrator == migrator, NotMigrator()); MOOLAH.setAuthorization(_migrator, enabled); emit MigratorAuthorization(_migrator, enabled); }</pre>
Description	However, the owner retains several ways to make the call revert, and the deadline is only a timestamp that never freezes the owner's CDP position or removes their control after it passes. 1. The same address configured as YIELD_ACCOUNT_OWNER is also the OWNER of the YieldAccount. The routed migration requires the YieldAccount to have authorized the migrator on Moolah, which _yieldAccount() checks before proceeding. The setMigratorAuthorization() function on the YieldAccount accepts calls from OWNER and has no pause restriction, so the owner can disable that authorization at any time, which forces forceMigrate() to revert. The MANAGER role can re enable the authorization, but the owner can disable it again and front run each attempt, so only an atomic bundle that re authorizes and calls forceMigrate() in one transaction reliably defeats this. 2. The migrator reads the live CDP debt and collateral on every call and requires the debt to be > 0. The owner can front-run a forced migration to repay their CDP debt down to 0 so that the call reverts with no debt to migrate, and then backrun it to re-borrow the lisUSD position. Again he can repeat this front-run.

	Impact The impact is that the owner can stall the forced CDP to Moolah migration indefinitely, which defeats the intended guarantee of forceMigrate().
Recommendation	Consider removing the owner's ability to revoke the migrator authorization for the routed account by restricting setMigratorAuthorization() to MANAGER or DEFAULT_ADMIN rather than allowing OWNER, so the owner cannot toggle it off before each forced attempt. Additionally, consider disallowing CDP debt creation when force migration is required.
Status	Acknowledged, ListaDao has mentioned that they will perform an atomic "re-authorise + forceMigrate" bundle, although this does not block the owner from still front-running this bundle and performing the blockage. An alternative would be to only allow authorization by the yield account owner before the current deadline has past.

[M02] Full principal exit through the exact amount in withdraw skips the forced skim and strands sub-threshold yield	
Contract	YieldAccount.sol
Severity	Medium
Description	Description The YieldAccount.withdraw() function decides whether to force a skim based on the exitAll flag, and that flag is set to true only when bnbAssets == type(uint256).max. An owner can fully exit their principal a second way, by passing bnbAssets == principalBnb, the exact principal value. In that case exitAll is false, so withdraw() calls _skim(false), and the skim is skipped whenever the surplus yield in BNB value is < minSkimBnb. This breaks the invariant documented in withdraw() that a full exit must force a skim so that a sub-threshold residual is not left uncollected. <pre> JavaScript function withdraw(uint256 bnbAssets, address receiver) external onlyOwner nonReentrant whenNotPaused { require(bnbAssets > 0, ZeroAmount()); require(receivers.contains(receiver), NotReceiver()); _sync(); bool exitAll = bnbAssets == type(uint256).max; // a full exit force-skims: minSkimBnb must not let a sub-threshold residual leave with the owner </pre>

```

        _skim(exitAll);

        uint256 coll = MOOLAH.position(marketId,
address(this)).collateral;
        uint256 slisOut;
        uint256 bnbCharged;
        if (exitAll) {
            slisOut = coll;
            bnbCharged = principalBnb;
            principalBnb = 0;
        } else {
            require(bnbAssets <= principalBnb, ExceedsPrincipal());
            slisOut = STAKE_MANAGER.convertBnbToSnBnb(bnbAssets);
            require(slisOut <= coll, InsufficientCollateral());
            bnbCharged = bnbAssets;
            principalBnb -= bnbAssets;
        }
        require(slisOut > 0, ZeroAmount());
    }

```

Impact

The withdrawal payout is capped at principal, since `slisOut = convert(bnbAssets)`, so the owner does not directly receive the residual. However, after this exit `principalBnb` becomes 0 and a sub-threshold yield residual $R = \text{collateralValue} - \text{principal}$, with $R < \text{minSkimBnb}$, remains stranded in the account.

- The permissionless `skim()` cannot collect it, because `_skim(false)` returns 0 for amounts $< \text{minSkimBnb}$ and `skim()` then reverts with `NothingToSkim`.
- The owner's own `withdraw(max)` also cannot sweep it, because the forced skim first sends all of R to the treasury and leaves 0 collateral, after which `slisOut = 0` reverts.
- The residual only becomes collectable once `slisBNB` appreciation, or residuals accumulated across later cycles, push the value back above `minSkimBnb`.

There is a second smaller impact where because R is left un-skimmed, the owner can borrow against it, since `borrow()` also only performs `_skim(false)`. If the resulting loan is never repaid, liquidation consumes R and the treasury loses it. Every path here is bounded by `minSkimBnb` per account per cycle, so the effect is dust level.

Recommendation

Consider treating any withdrawal that reduces principal to 0 as a full exit for skim purposes, rather than keying only on `bnbAssets == type(uint256).max`. For example, compute `fullExit = exitAll || bnbAssets >= principalBnb` and pass `fullExit`

	into _skim, so the exact principal path forces a skim in the same way the max path does.
Status	Fixed as recommended, now computes <code>exitAll = bnbAssets == type(uint256).max bnbAssets == principalBnb</code> , ensuring force skimmed is performed if principalBNB is zeroed out. (TBD, can we leave a very small portion of principalBNB to prevent force skim, will that cause any issues)?

[M03] Insufficient lisUSD liquidity in Moolah makes the full-debt flash-loan migration unexecutable for YIELD_ACCOUNT_OWNER currently

Contract	PositionMigrator.sol
----------	----------------------

Severity	Medium
----------	--------

	Description The migration flow flash-loans the full CDP debt in a single transaction via MOOLAH.flashLoan(LISUSD, cdpDebt, data), repays the CDP, releases and supplies the collateral to Moolah, and re-borrows the same amount to repay the flash loan. The critical detail is the ordering inside onMoolahFlashLoan: Moolah transfers the flash-loaned cdpDebt out first, then the re-borrow (repaid, approximately equal to cdpDebt) is drawn, and only after the callback returns does Moolah pull the flash loan back. So the flash loan and the re-borrow are outstanding against Moolah's lisUSD balance at the same time. That means the requirement is not two independent "at least cdpDebt" checks. It is: 1. Physical balance (about 2x the debt). At the moment of the re-borrow, Moolah's lisUSD token balance must cover both the still-outstanding flash loan and the new borrow, i.e. $cdpDebt + repaid$, which is approximately $2 * cdpDebt$. The flash-loaned tokens are not returned until after the borrow, and the portion sent to the CDP is burned on repayment, so the borrow draws entirely fresh tokens from Moolah rather than reusing the flash-loaned ones. 2. Market borrowable (about 1x the debt). The re-borrow is additionally capped by the target slisBNB/lisUSD market's accounting liquidity ($totalSupplyAssets - totalBorrowAssets$), which must be at least repaid, roughly cdpDebt. Because the target market's idle liquidity is itself part of Moolah's physical balance, the practical requirement is that Moolah hold about $2 * cdpDebt$ in idle lisUSD, of which at least about cdpDebt must be borrowable specifically in the target market.
Description	

On-chain state (BSC mainnet, block approximately 120,830,169)

For the only migratable account, YIELD_ACCOUNT_OWNER
(0x0966602E47F6a3CA5692529F1D54EcD1d9B09175):

- The CDP debt to repay, read from [Interaction.borrowed\(ceBNB, owner\)](#) 19,142,363.26 lisUSD (this is the flash-loan amount).
- [Moolah's lisUSD balance](#), `lisUSD.balanceOf(Moolah)` (the flash-loan capacity): 8,741,759.58 lisUSD.
- Target market
0x7fe248d8459a88e50e8582c71219edbce1079437e58190aeab41ac503694f
0a5: 27,465,168.48 supplied, 19,350,373.50 borrowed, leaving 8,114,794.98 borrowable.

Impact

Because neither condition is met, and the corrected requirement makes the gap larger than a single 1x shortfall suggests:

- Flash loan reverts first. MOOLAH.flashLoan transfers `cdpDebt` up front, and $19,142,363.26 > 8,741,759.58$, so the transfer reverts immediately and the migration never proceeds, a shortfall of about 10,400,604 lisUSD against a single 1x need.
- The re-borrow leg would also fail if reached. Re-borrowing 19,142,363.26 against 8,114,794.98 borrowable trips `INSUFFICIENT_LIQUIDITY`, a shortfall of about 11,027,568 lisUSD.

There is no partial-migration fallback, because the flow is atomic and all or nothing, so the full 19,142,363.26 lisUSD must be simultaneously available as flash-loan liquidity and, together with the re-borrow, within Moolah's physical balance and the market's borrow capacity. The failure is safe, since it only reverts and does not lose funds, but the feature cannot be used at the present state.

`Interaction.borrowed` is read before `drip`, and the migrator calls `drip` first, so the true debt and required flash loan are marginally higher, which slightly widens both shortfalls. For context, the position is heavily over-collateralized: `Interaction.locked(ceBNB, owner)` is 181,999.99 ceBNB units, about 182,000 BNB, backing the 19,142,363.26 lisUSD debt, an LTV of roughly 17.5% against the 85% LLTV.

Recommendation	As an alternative or complement, support partial or tranching migration so the position can be moved in increments that fit current liquidity, or gate the migration behind an explicit liquidity precondition and coordinate a temporary lisUSD supply top-up to the slisBNB/lisUSD market before executing. However, this might require a large amount of lisUSD balance available, if not, repayments of the CDP position needs to be performed to burn lisUSD to facilitate the transfer.
Status	Acknowledged, the migration remains atomic full-debt (flash-loan + re-borrow), so it still requires $-2\times$ the debt as idle lisUSD in Moolah and $-1\times$ borrowable in the target market. No partial/tranching path or liquidity precondition was added. Failure is safe (reverts, no fund loss). Resolution thus depends on a coordinated lisUSD supply top-up or CDP repayment before executing, per the recommendation.

[L01] Liquidation seizure valued at the current exchange rate over-charges principal and inflates the treasury yield claim

Contract	PositionMigrator.sol & YieldAccount.sol
Severity	Low
Description	Description When the YieldAccount position is liquidated, Moolah reduces the account's collateral directly and the YieldAccount receives no synchronous notification. Its principal is reconciled by <code>_sync</code>, which only runs on the next interaction with the account, meaning the next call to <code>borrow</code>, <code>withdraw</code>, <code>skim</code>, <code>depositBnb</code>, <code>depositSlisBnb</code>, or <code>sync</code>. <pre> JavaScript function _sync() internal { uint256 coll = MOOLAH.position(marketId, address(this)).collateral; uint256 tracked = trackedCollateral; if (coll < tracked) { uint256 seizedValue = STAKE_MANAGER.convertSnBnbToBnb(tracked - coll); uint256 p = principalBnb; if (seizedValue > 0 && p > 0) { uint256 newPrincipal = p > seizedValue ? p - seizedValue : 0; emit PrincipalSynced(p, newPrincipal); principalBnb = newPrincipal; } } } </pre>

```

    if (coll != tracked) trackedCollateral = coll;
}

```

_sync charges the owner's principalBnb by converting the seized slisBNB amount, computed as the difference between the tracked collateral and the current collateral, into BNB using the StakeManager exchange rate at the moment _sync executes. The slisBNB to BNB rate only rises over time because of staking accrual. As a result, any delay between the liquidation and the _sync call values the seized units at a higher rate than the rate that existed at liquidation, so principalBnb is reduced by more than the BNB value that actually left the position.

Impact

Because the claimable yield is defined as the current collateral value minus principal, this excess reduction of principal raises the treasury's claimable yield above the true figure.

Nothing forces _sync to run promptly, so value therefore shifts from the owner's principal to the treasury across the delay window. The size of the shift is bounded by the rate growth during the delay, and the drift is correctable by anyone calling sync, so the impact is limited but the accounting is directionally wrong.

Recommendation

Status

Acknowledged. No code change. _sync still converts the seized slisBNB at the live rate at _sync time, so a delay between liquidation and sync shifts value from owner principal to the treasury's yield claim. This is bounded by rate growth over the delay window and correctable by anyone calling sync() so a keeper sync is expected to be performed shortly after liquidation.

[L02] borrow is not strictly capped at principal because sub-threshold yield is not skimmed

Contract

YieldAccount.sol

Severity

Low

Description

Description

The YieldAccount contract documents that skimming runs before every borrow so that borrowing is limited to the recorded BNB principal, with Moolah's own health check enforcing the cap. In practice borrow() calls _skim(false), which skips the skim whenever the claimable yield value is below minSkimBnb.

JavaScript

```

function _skim(bool force) internal returns (uint256) {
    MOOLAH.accrueInterest(marketParams);
}

```

```
uint256 claimable = claimableYield();
(uint256 amount, uint256 bnbValue) = _skimmable();
if (amount == 0 || (!force && bnbValue < minSkimBnb)) return 0;
```

When the excess yield above principal is under this threshold, it stays in the collateral position at the time the borrow executes. Moolah's health check then prices the loan against the full collateral, which equals principal plus the un-skimmed yield, rather than against principal alone.

Impact

As a result, the owner can borrow against principal plus up to minSkimBnb of yield that belongs to the treasury. The over-borrow is bounded by minSkimBnb, which is 0.005 BNB in the deploy config, and with the market LLTV of 85% the extra borrowing capacity is roughly $0.005 * 0.85$ of loan value per position. If the position later defaults and is liquidated, that treasury yield is consumed to cover the owner's debt instead of being collected by the treasury.

Note that the Moolah's minimum loan value does not prevent this. That check is a floor on total debt for dust prevention, not a ceiling. The owner borrows the maximum in a single call, so the marginal extra capacity from the un-skimmed yield is included inside a large loan whose total easily exceeds the minimum, and the floor is satisfied by the principal-backed portion.

Consider force-skim by calling `_skim(true)` before borrowing so the collateral equals principal at the time of the health check, accepting that this reintroduces small skims that minSkimBnb was meant to avoid.

Recommendation

Alternatively, if the dust-skim churn is not acceptable, update the invariant documentation to state that the borrow ceiling is principal plus up to minSkimBnb rather than principal alone.

Status

Fixed documented per the recommendation's second option. Invariant #2 (YieldAccount.sol:31–33) now states the borrow ceiling is "principal plus at most minSkimBnb", since borrow() uses the non-forcing `_skim(false)`.

Additionally the new MAX_MIN_SKIM_BNB = 1 ether cap bounds the leak to $\leq 1 \text{ BNB} \times \text{LLTV}$ per position.

[L03] Owner can dodge treasury yield capture by exiting around slisBNB appreciation events

Contract YieldAccount.sol

Severity Low

Description

The YieldAccount is built so that slisBNB exchange rate appreciation on the position accrues to the treasury, but that only holds while the owner's collateral actually sits inside the account, and the account never locks the owner in. withdraw() is instant and unlocked, and on exit it force-skims only the appreciation accrued in-account up to that instant, so an owner who exits before any appreciation leaves nothing behind for the treasury, which is correct.

JavaScript

```
/// @param force ignore minSkimBnb (full-exit path)
function _skim(bool force) internal returns (uint256) {
    MOOLAH accrueInterest(marketParams);
    uint256 claimable = claimableYield();
    (uint256 amount, uint256 bnbValue) = _skimmable();
    if (amount == 0 || (!force && bnbValue < minSkimBnb)) return 0;

    address to = treasury;
    uint256 balanceBefore = IERC20(TOKEN).balanceOf(to);
    PROVIDER.withdrawCollateral(marketParams, amount, address(this),
to);
    require(IERC20(TOKEN).balanceOf(to) >= balanceBefore + amount,
WithdrawShortfall());
    trackedCollateral = MOOLAH.position(marketId,
address(this)).collateral;

    emit YieldSkimmed(amount, bnbValue, claimable);
    return amount;
}
```

Description

Impact

The weakness comes from how appreciation is realized. It is not continuous. In ListaStakeManager the rate stays flat between discrete, BOT-controlled compoundRewards() calls, each of which steps the rate up via totalDelegated += totalProfit, roughly once per cycle. A rational owner can front-run these steps. They withdraw from the YieldAccount just before a compound, which is a possible exit because no in-account yield exists yet, hold the slisBNB in the CDP or a plain wallet across the rate step where the gain accrues to them since there is no skim, and then re-deposit or re-migrate afterward through depositBnb(), depositSlisBnb(), migratePosition, or forceMigrate. The re-entry records principalBnb at the new appreciated value, resetting the owner's baseline upward. The treasury is denied the appreciation it would have captured had the owner stayed.

JavaScript

```

/**
 * @dev Allows bot to compound rewards
 */
function compoundRewards() external override whenNotPaused
onlyRole(BOT) {
    if (totalDelegated == 0) revert ErrorsLib.NotEnoughBnb();

    uint256 totalBNBInValidators = getTotalBnbInValidators();
    if (totalBNBInValidators + undelegatedQuota <=
totalDelegated) {
        revert ErrorsLib.NotEnoughBnb();
    }
    uint256 totalProfit = totalBNBInValidators +
undelegatedQuota - totalDelegated;
    uint256 fee =
SLisLibrary.calculateFeeFromDailyProfit(totalProfit, synFee,
TEN_DECIMALS);

    totalDelegated += totalProfit;
    uint256 slisBNBAmount = convertBnbToSnBnb(fee);
    if (slisBNBAmount > 0) {
        ISLisBNB(slisBnb).mint(revenuePool, slisBNBAmount);
    }

    _burnRefundSlisBnb();
    emit RewardsCompounded(fee);
}

```

The impact is a denial of the protocol's intended durable yield capture rather than a direct loss. forceMigrate is only a partial defense, because it can pull a CDP position back only after migrationDeadline and cannot reach slisBNB held in a plain wallet, and adding skim or sync logic to the migrator does not help because the appreciation occurred outside the account. The owner also collects slisBNBx during the exit window, which compounds the separate owner-controllable delegateSlisBNBx concern.

The primary mitigations are operational. Pair a keeper skim() of the YieldAccount with each compoundRewards, or run it in the block immediately after, and avoid a publicly predictable compound schedule.

Recommendation

Alternatively, since the MANAGER role controls the whitelisted receivers that can receive the exited collateral, if this kind of bypass happen, they can extract the earned yield before redepositing back into the yield account.

Status

Acknowledged, no code change, mitigation is operational (keeper skim() paired with compoundRewards(), non-predictable compound schedule, and MANAGER-controlled receiver whitelist as backstop).

[I01] Provider divergence freezes withdraw and skim when Moolah reregisters a different provider	
Contract	YieldAccount.sol
Severity	Informational
Description	Description The YieldAccount contract stores its PROVIDER as an immutable address set at construction, while the provider registered for the market inside Moolah is mutable and can be changed by a trusted admin via setProvider. The deposit path re-reads Moolah's currently registered provider and reverts cleanly if it no longer matches the account's immutable PROVIDER. <pre>JavaScript ISlisBnbProvider public immutable PROVIDER;</pre> Impact However, the withdraw and _skim paths call the immutable PROVIDER directly without any equivalent check. If the market's registered provider is changed away from the account's PROVIDER, Moolah will reject collateral withdrawals routed through the now stale provider. Because withdraw and skim are the only ways funds leave the position, both the owner's principal and the treasury's yield become frozen until the original provider is registered again or the account is upgraded through UUPS. Recommendation Consider removing the owner's ability to revoke the migrator authorization for the routed account by restricting setMigratorAuthorization() to MANAGER or DEFAULT_ADMIN rather than allowing OWNER, so the owner cannot toggle it off before each forced attempt. Additionally, consider disallowing CDP debt creation when force migration is required.
Status	Acknowledged, if an admin re-registers a different provider, both paths freeze until the original is restored or the account will be upgraded (UUPS).

[I02] Missing event emission for state variable assignments in initialize()	
Contract	YieldAccount.sol
Severity	Informational
Description	Description The YieldAccount.initialize() function sets several configuration state variables without emitting the events that the contract already defines for those same values. In particular, treasury and minSkimBnb are assigned during initialization but

no SetTreasury or SetMinSkimBnb event is emitted, even though the dedicated setters setTreasury() and setMinSkimBnb() do emit those events on every later update. The market configuration values assigned at initialization are also written silently.

JavaScript

```
marketParams = _marketParams;  
marketId = id;  
treasury = _treasury;  
minSkimBnb = _minSkimBnb;
```

Impact

This creates an inconsistency between how the initial values and the later values are surfaced. Off chain services, indexers, and monitoring tools that reconstruct the contract configuration by listening to events will never observe the initial treasury and minSkimBnb values, so their view of the configuration starts incomplete until the first setter call happens to fire. There is no direct fund loss, but the gap reduces transparency and can cause off chain accounting or alerting systems to operate on stale or missing configuration data.

Recommendation

Consider emitting the same events inside initialize() that the corresponding setters emit, so that the initial state is observable on chain from the moment the contract is initialized. Emit SetTreasury for the initial treasury value and SetMinSkimBnb for the initial minSkimBnb value, and add an event for the initial market configuration assignment. Aligning the initializer with the setters ensures every configuration value has a consistent event trail for off chain consumers.

Status

Fixed, initialize() now emits SetTreasury and SetMinSkimBnb, matching the setters.

[I03] View function inconsistencies

Contract

YieldAccount.sol

Severity

Informational

Description

Description

The skimmable() and previewSkim() view functions expose the internal _skimmable() calculation, which derives the health capped removable collateral from the position's debt. In a view context there is no way to call accrueInterest, so _skimmable() reads the stored market totals and converts the position's borrow shares to assets using those stale totals. This excludes any interest that has accrued since Moolah last updated the market. Understating the debt understates the minimum required collateral, which in turn overstates the removable and skimmable amount whenever the position carries outstanding debt and sits close to the LLTV cap.

JavaScript

```
/// @dev what a skim in this block could take, health-capped
function skimmable() public view returns (uint256 slisAmount,
uint256 bnbValue) {
    return _skimmable();
}

/// @dev backend dry-run. `claimable` is the accounting figure;
`skimmableBnb` is what a skim in
/// this block can actually take. They differ when debt has
grown against the health cap.
function previewSkim() external view returns (uint256 claimable,
uint256 skimmableSlis, uint256 skimmableBnb) {
    claimable = claimableYield();
    (skimmableSlis, skimmableBnb) = _skimmable();
}
```

Separately, `claimableYield()` does not price debt at all. It only compares the collateral value in BNB against the recorded principal, so it reports the full accounting surplus and ignores both outstanding debt and the health cap. As a result it can report claimable yield that a skim cannot actually remove when debt constrains the position. This is the same gap that `previewSkim()` surfaces as the difference between its `claimable` and `skimmableBnb` outputs.

JavaScript

```
function claimableYield() public view returns (uint256) {
    uint256 value = STAKE_MANAGER.convertSnBnbToBnb(collateral());
    uint256 principal = principalBnb;
    return value > principal ? value - principal : 0;
}
```

Impact

The actual `skim()` transaction accrues interest before computing the same values, so the on chain result is correct and self correcting. The overstatement exists only in the values returned to off chain callers. `previewSkim()` describes its `skimmableBnb` output as the amount that a skim in this block can actually take, which is optimistic in the case where the position is near the cap, because the view uses pre accrual debt while the transaction uses post accrual debt. A backend that sizes a skim from these views can therefore request more than the transaction will grant.

This also makes `_skimmable()` inconsistent with the other views. `borrowable()` and `withdrawable()` price the debt through `expectedBorrowAssets`, which simulates pending interest and is interest accurate, while `_skimmable()` does not.

Recommendation	When the position has no debt (<code>borrowShares == 0</code>), <code>removable = coll</code> , so the skimmable amount equals the surplus and is interest independent. The discrepancy only appears when there is outstanding debt near the cap. There is no loss of funds, because the effect is limited to inaccurate view output.
	Consider pricing the debt inside <code>_skimmable()</code> using <code>expectedBorrowAssets</code> from <code>MoolahBalancesLib</code> , the same source that <code>borrowable()</code> and <code>withdrawable()</code> already use.
	Inside <code>skim()</code> the calculation runs after interest has been accrued, so the value is identical and there is no change to on chain behavior, while the <code>skimmable()</code> and <code>previewSkim()</code> views become interest accurate and consistent with the other views.
	If <code>claimableYield()</code> is meant to remain a pure accounting figure, keep it as is but document clearly that it ignores debt and the health cap, and point integrators to the skimmable output for the amount that a skim can actually remove.
Status	Fixed, <code>_skimmable()</code> now prices debt via <code>MOOLAH.expectedBorrowAssets()</code> , the interest-accurate source <code>borrowable()/withdrawable()</code> already use, so <code>skimmable()/previewSkim()</code> no longer overstate the skim near the health cap.

[I04] Routed migration for YIELD_ACCOUNT_OWNER accepts <code>isBnb == false</code> as a latent, unused path	
Contract	<code>PositionMigrator.sol</code>
Severity	Informational
Description	Description The <code>PositionMigrator</code> routes <code>YIELD_ACCOUNT_OWNER</code>'s migration into the <code>YieldAccount</code> through <code>migratePosition()</code>, <code>_migrate()</code>, and <code>onMoolahFlashLoan()</code>, and it accepts <code>isBnb == false</code> for this owner even though <code>forceMigrate()</code> always uses <code>isBnb == true</code> and the only position this owner actually holds on-chain is a BNB CDP position. <pre> JavaScript function forceMigrate(MarketParams calldata marketParams, uint256 minSlisBnb) external nonReentrant onlyRole(BOT) returns (uint256) { uint256 deadline = migrationDeadline; require(deadline != 0, "deadline not set"); require(block.timestamp >= deadline, "migration window open"); // the same whitelist gate as the voluntary path require(whitelist.contains(YIELD_ACCOUNT_OWNER), "not whitelisted"); </pre>

```

        uint256 migrated = _migrate(marketParams, YIELD_ACCOUNT_OWNER,
        true, minSlisBnb);

        emit ForcedMigration(msg.sender, migrated);

        return migrated;
    }

```

On-chain reads confirm the owner has zero slisBNB CDP debt and zero slisBNB CDP collateral (borrowed and locked are both 0 on the slisBNB ilk) alongside a large BNB CDP position. Because of this, an `isBnb == false` call currently reverts at the "no debt to migrate" check and does nothing.

Impact

The path is latent rather than active. For the routed account, `_yieldAccount` pins the target to the slisBNB/lisUSD yield-account market, so `isBnb == false` can only ever read the slisBNB ilk and never the BNB ilk, and it releases collateral through `slisBnbProviderCDP.releaseFor` rather than through `bnbProvider`. If the owner opens a slisBNB CDP position in the future, a voluntary `isBnb == false migratePosition()` would move that slisBNB denominated, yield bearing collateral into the `YieldAccount`, record its current BNB value as principal, and send all future exchange rate growth to the treasury through skimming.

Recommendation

Consider constraining the routed path (the branch where `onBehalf == YIELD_ACCOUNT_OWNER`) to `isBnb == true`, so the migrator's exposed behavior matches the owner's actual position and the unused slisBNB ilk branch is removed.

Status

Fixed, `_migrate()` now requires `onBehalf != YIELD_ACCOUNT_OWNER || isBnb`, constraining the routed account to the BNB path and removing the dead slisBNB-ilk branch. Placed ahead of the CDP reads so the "no debt to migrate" revert cannot mask it.

[I05] Missing `borrowShares > 0` guard in the partial branch of `repay()`

Contract

PositionMigrator.sol

Severity

Informational

Description

Description

In `YieldAccount.repay()`, the full repayment branch (called with `assets == type(uint256).max`) explicitly requires `borrowShares > 0` before proceeding, but the partial repayment branch (called with a specific amount) has no equivalent check.

JavaScript

```
function repay(uint256 assets) external nonReentrant returns
(uint256 repaidAssets, uint256 repaidShares) {
    MarketParams memory params = marketParams;
    IERC20 loanToken = IERC20(params.loanToken);

    if (assets == type(uint256).max) {
        MOOLAH.accrueInterest(params);
        Position memory pos = MOOLAH.position(marketId,
address(this));
        require(pos.borrowShares > 0, ZeroAmount());
        Market memory m = MOOLAH.market(marketId);
        uint256 owed =
uint256(pos.borrowShares).toAssetsUp(m.totalBorrowAssets,
m.totalBorrowShares);

        loanToken.safeTransferFrom(msg.sender, address(this), owed);
        loanToken.forceApprove(address(MOOLAH), owed);
        (repaidAssets, repaidShares) = MOOLAH.repay(params, 0,
pos.borrowShares, address(this), "");
    } else {
        require(assets > 0, ZeroAmount());
        loanToken.safeTransferFrom(msg.sender, address(this), assets);
        loanToken.forceApprove(address(MOOLAH), assets);
        (repaidAssets, repaidShares) = MOOLAH.repay(params, assets, 0,
address(this), "");
    }
}
```

Impact

When the position carries no debt, the partial branch does not reject the call on its own and instead depends on Moolah reverting the repay. The outcome is still safe because Moolah blocks a repay against a position with zero debt, so this is cosmetic. The concern is that the two branches behave inconsistently and the account does not fail on its own before reaching the external call. There is no fund risk, only a consistency and clarity gap.

Recommendation

Consider adding an explicit borrowShares > 0 check to the partial branch so both branches reject a repay against a position with no outstanding debt directly, if not, to be consistent, we can instead rely on the downstream Moolah revert.

Status

Fixed, the partial branch now requires borrowShares > 0 before pulling funds.

[I06] minLoan inverse price scaling pushes positions into a band where partial repayment reverts

Contract

YieldAccount.sol

Severity

Informational

Description

Description

Per the following [PR](#), Moolah enforces a minimum remaining debt on every borrow and repay through `_checkBorrowAssets`, which passes only when the account's remaining borrow assets are 0 or $\geq \text{minLoan}(\text{marketParams})$. `minLoan` is denominated in loan token (lisUSD) units and is computed as $\text{minLoanValue} * 10^{\text{decimals}} / \text{price}$, where $\text{price} = \text{oracle.peek}(\text{loanToken})$ is the lisUSD price. Because the formula divides by the lisUSD price, the floor expressed in lisUSD rises when lisUSD trades below peg.

Production `minLoanValue` is $15e8$, representing \$15. At a \$1.00 price the floor is 15 lisUSD, and at \$0.80 it rises to 18.75 lisUSD. As such, if the `YieldAccount` has a borrow position for the defined market (lisUSD) whose total debt was legal when opened, for example 16 lisUSD, which is above the 15 floor, silently falls into the (0, `minLoan`) band once the price moves. Any partial repay then leaves the remaining debt inside (0, `minLoan`) and reverts with `REMAIN_BORROW_TOO_LOW`. A `MANAGER` raising `minLoanValue` through `setMinLoanValue` produces the same effect.

Impact

Full repayment via `type(uint256).max` always still works, because it drives the remaining `borrowShares` to 0 and `_checkBorrowAssets` short-circuits to true at 0, so debt is never permanently trapped. What is lost is the ability to partially deleverage, and this occurs during a `depeg` exactly when a borrower would most want to pay part of the debt down to restore health or free collateral. Small new borrows below the raised floor are likewise blocked.

This is Moolah wide behavior inherited by `YieldAccount.repay` through its partial repay path, not a defect introduced by the account. The `minLoanValue` raise path is unconditional. The `depeg` path is enabled at the contract level, because `OracleAdaptor.peek` routes lisUSD through a passthrough to `RESILIENT_ORACLE.peek(lisUSD)` with no hardcoded \$1 peg, so it fires whenever the resilient oracle reports the live below peg lisUSD price. If that oracle instead pins lisUSD to \$1, only the `minLoanValue` path applies.

Recommendation	Consider documenting this issue and perform full repayment when required.
-----------------------	---

Status	Acknowledged. Moolah-wide behaviour inherited via <code>repay()</code> 's partial path, not a <code>YieldAccount</code> defect. Mitigation is to perform a full repayment (<code>type(uint256).max</code>), which always drives remaining shares to 0 and bypasses the floor.
---------------	---

[I07] repay returns the contract's entire loanToken balance to the caller

Contract	<code>YieldAccount.sol</code>
-----------------	-------------------------------

Severity	Informational
-----------------	---------------

Description	Description
--------------------	-------------

At the end of `YieldAccount.repay()`, any remaining `loanToken` (`lisUSD`) balance held by the contract is transferred to `msg.sender`. In normal operation the account never holds `lisUSD`, because borrowed funds are sent straight to the receiver and a repay nets the balance back to approximately 0, so this sweep only ever returns the caller's own rounding dust.

JavaScript

```
function repay(uint256 assets) external nonReentrant returns
(uint256 repaidAssets, uint256 repaidShares) {
    MarketParams memory params = marketParams;
    IERC20 loanToken = IERC20(params.loanToken);

    if (assets == type(uint256).max) {
        MOOLAH accrueInterest(params);
        Position memory pos = MOOLAH.position(marketId,
address(this));
        require(pos.borrowShares > 0, ZeroAmount());
        Market memory m = MOOLAH.market(marketId);
        uint256 owed =
uint256(pos.borrowShares).toAssetsUp(m.totalBorrowAssets,
m.totalBorrowShares);

        loanToken.safeTransferFrom(msg.sender, address(this), owed);
        loanToken.forceApprove(address(MOOLAH), owed);
        (repaidAssets, repaidShares) = MOOLAH.repay(params, 0,
pos.borrowShares, address(this), "");
    } else {
        require(assets > 0, ZeroAmount());
        loanToken.safeTransferFrom(msg.sender, address(this), assets);
        loanToken.forceApprove(address(MOOLAH), assets);
        (repaidAssets, repaidShares) = MOOLAH.repay(params, assets, 0,
address(this), "");
    }
}
```

Impact

However, `repay()` is permissionless, so if `lisUSD` is ever donated or mistakenly sent to the account, an arbitrary caller can invoke `repay(1)` and collect that stray balance through the leftover sweep. This does not expose protocol funds, since the account is not designed to custody `lisUSD`, but any stray `lisUSD` sitting in the contract is claimable by anyone who calls `repay`.

Recommendation

Consider refunding only the amount left over from this specific `repay` call rather than the full contract balance. Record the `loanToken` balance before and after the `repay`, and return only that delta to `msg.sender`, so the refund is confined to the caller's own unused input and an unrelated caller cannot sweep donated or stray `lisUSD`.

	A future upgrade can be performed to retrieve any stray loan or donated tokens from the contract.
Status	Fixed, the refund is now pulled – repaidAssets (underflow-guarded by if (pulled > repaidAssets)) instead of sweeping the full balance, so donated/stray lisUSD is no longer claimable by an arbitrary caller performing small repayment.

[I08] All deposits into YieldAccount forfeit their slisBNB appreciation to the treasury post migration

Contract	YieldAccount.sol
Severity	Informational

Description	Description Every deposit into YieldAccount through depositBnb() or depositSlisBnb(), not just the originally migrated position, permanently gives up its future slisBNB exchange rate appreciation to the treasury. A deposit records principalBnb at the deposit time BNB value, so the owner keeps the BNB value of whatever they add, but all subsequent appreciation on that capital is skimmed to Lista in the same way as the rest of the account. Deposits are permissionless, and there is no per deposit opt out and no way to deposit while keeping the yield. This behavior is a deliberate and internally consistent design, since the account is a single purpose, BNB denominated vehicle built on one principalBnb baseline where any value above principal belongs to the treasury, and it is not an accounting bug.
	<pre>JavaScript /// @dev Follows MoolahVaultAccount (principal baseline + /// surplus-only harvest) with three deliberate /// divergences: `skim` is permissionless rather than /// BOT-gated, since protocol yield must stay /// collectable if a keyholder is unavailable and it can only /// ever reach `treasury`; there is no /// `emergencyWithdraw`, because there the corpus is the /// protocol's own and here it is the /// owner's; and there is no `increasePrincipal`, because /// unlike a plain share transfer into a /// vault, every inflow here has a recording entry point – /// `depositBnb` / `depositSlisBnb` are /// permissionless – so collateral supplied straight through /// the provider is a donation and is /// correctly treated as yield.</pre> Impact The account resembles an ordinary collateral wrapper but silently reassigns all appreciation to the treasury, so fresh capital added by mistake, or by operations tooling that treats it like a normal position, forfeits its yield. Additionally, for top ups

	performed by the yield account owner to improve health and avoid liquidation, the appreciation on that top up is also surrendered, although the principal stays withdrawable so it is not lost. Lastly, the behavior can be broader than the migration, because if the intent was only to capture appreciation on the migrated CDP position that was always BNB denominated, then allowing arbitrary future deposits to also feed the treasury exceeds that scope.
Recommendation	Consider improving the documentation so it clearly states that any capital held in this account, whether migrated or freshly deposited, forfeits its exchange rate appreciation to the treasury, and that the account is not intended for capital whose yield the owner wants to keep.
Status	Fixed, a @dev note now documents that every deposit via migrated position or fresh top-up, including a health top-up, forfeits its appreciation to the treasury while principal stays withdrawable.

[I09] Missing future timestamp check in extendMigrationDeadline lets a past deadline pass as an extension	
Contract	PositionMigrator.sol
Severity	Informational
Description	Description The PositionMigrator.extendMigrationDeadline() function only requires that the new deadline is greater than the current one, checking <code>deadline > old</code>, and it never requires <code>deadline > block.timestamp</code>. When the existing deadline is already in the past, the MANAGER can set a new deadline that is still in the past, satisfying <code>old < deadline <= now</code>. <pre> JavaScript /// @dev MANAGER-callable because it can only ever delay a forced migration, never advance one function extendMigrationDeadline(uint256 deadline) external onlyRole(MANAGER) { uint256 old = migrationDeadline; require(old != 0, "deadline not set"); require(deadline > old, "not an extension"); migrationDeadline = deadline; emit MigrationDeadlineChanged(old, deadline); } </pre> Impact Because forceMigrate becomes callable once <code>block.timestamp >= deadline</code>, moving the deadline to another already passed value does not close the migration window again, so forceMigrate remains callable. The result is that a call which is named and

	intended to delay the forced migration can be a no operation for the window while appearing to extend it. This does not cause loss of funds, but it creates a gap between what the function name and intent imply and what actually happens, which is confusing for monitoring and operations that assume an extension always postpones the forced migration.
Recommendation	Consider requiring the new deadline to be in the future in extendMigrationDeadline() by adding a check such as deadline > block.timestamp in addition to the existing deadline > old check.
Status	Fixed, extendMigrationDeadline() now requires deadline > block.timestamp, and setMigrationDeadline() requires deadline == 0 deadline > block.timestamp. A past value can no longer arm forceMigrate in the same block.

[I10] Unnecessary claimableYield() computation in _skim wastes gas on the early return path	
Contract	YieldAccount.sol
Severity	Informational
Description	Description The YieldAccount._skim() function computes claimableYield() and stores the result in a local variable near the start of the function, before it reaches the early return guard that exits when there is nothing to skim, meaning when the skimmable amount is 0 or, on the non forced path, when the skimmable BNB value is below minSkimBnb. <pre> JavaScript // @param force ignore minSkimBnb (full-exit path) function _skim(bool force) internal returns (uint256) { MOOLAH accrueInterest(marketParams); uint256 claimable = claimableYield(); (uint256 amount, uint256 bnbValue) = _skimmable(); if (amount == 0 (!force && bnbValue < minSkimBnb)) return 0; address to = treasury; uint256 balanceBefore = IERC20(TOKEN).balanceOf(to); PROVIDER.withdrawCollateral(marketParams, amount, address(this), to); require(IERC20(TOKEN).balanceOf(to) >= balanceBefore + amount, WithdrawShortfall()); trackedCollateral = MOOLAH.position(marketId, address(this)).collateral; emit YieldSkimmed(amount, bnbValue, claimable); return amount; } </pre> Impact

	That stored value is used only to populate the claimableAtCall field of the YieldSkimmed event, which is emitted at the end of the function on the path where a skim actually happens. When the early return is taken, the event is never emitted, so the claimableYield() result is discarded and the work is wasted. claimableYield() is not free, since it performs an external convertSnBnbToBnb call on the stake manager and reads the position collateral from Moolah, so every skim call that ends in the early return, including routine calls where nothing is currently skimmable, pays for a computation whose only consumer is skipped.
Recommendation	Consider moving the claimableYield() computation to after the early return guard, so it runs only when a skim is actually going to proceed and the YieldSkimmed event will be emitted.
Status	Fixed, claimableYield() is now computed after the early-return guard, so the external convertSnBnbToBnb call and collateral read only run when a skim proceeds.

[I11] Missing upper bound threshold for setMinSkimBnb

Contract	YieldAccount.sol
Severity	Informational
Description	Description The setMinSkimBnb function accepts any value with no lower or upper bound, and initialize also assigns minSkimBnb without any range check, so MANAGER can set it arbitrarily high. <pre> JavaScript // @param force ignore minSkimBnb (full-exit path) function _skim(bool force) internal returns (uint256) { MOOLAH.accrueInterest(marketParams); uint256 claimable = claimableYield(); (uint256 amount, uint256 bnbValue) = _skimmable(); if (amount == 0 (!force && bnbValue < minSkimBnb)) return 0; address to = treasury; uint256 balanceBefore = IERC20(TOKEN).balanceOf(to); PROVIDER.withdrawCollateral(marketParams, amount, address(this), to); require(IERC20(TOKEN).balanceOf(to) >= balanceBefore + amount, WithdrawShortfall()); trackedCollateral = MOOLAH.position(marketId, address(this)).collateral; emit YieldSkimmed(amount, bnbValue, claimable); return amount; } </pre>

Impact

A very large minSkimBnb causes two harmful effects.

1. It widens the borrow cap leak. The borrow function calls the non forcing skim, which skips skimming whenever the yield value is below minSkimBnb. Any accrued yield below that threshold therefore stays in the collateral, and Moolah's health check lets the owner borrow against the principal plus that un-skimmed yield. Since skim() is gated by the same threshold, no one can skim until the yield reaches minSkimBnb, so with a large value the entire accrued yield sits un-skimmable and backs the owner's borrow. If the owner then defaults, that treasury owned yield is lost together with the seized collateral.
2. It blocks treasury collection. While the accrued yield value stays below minSkimBnb, skim() reverts with NothingToSkim. The only path that force skims and ignores minSkimBnb is the full exit branch of withdraw, which is restricted to the owner and has no permissionless equivalent. As a result, for as long as minSkimBnb sits above the accrued yield, the treasury cannot collect any yield through skim(), and collection depends entirely on the owner deciding to fully exit.

This in differs with the contract's stated goal that no single MANAGER key can shrink the protocol's yield claim. MANAGER cannot change claimableYield directly, so the accounting figure is untouched, but through setMinSkimBnb a single MANAGER key can lock the yield and expose it to the owner's borrowing and to liquidation, which impairs how much of the claim is actually realizable.

Recommendation

Consider bounding setMinSkimBnb with a realistic upper bound threshold, if not, ensure that the MANAGER role is fully trusted.

Status

Fixed. New MAX_MIN_SKIM_BNB = 1 ether constant enforced in both initialize() and setMinSkimBnb() via InvalidMinSkim.